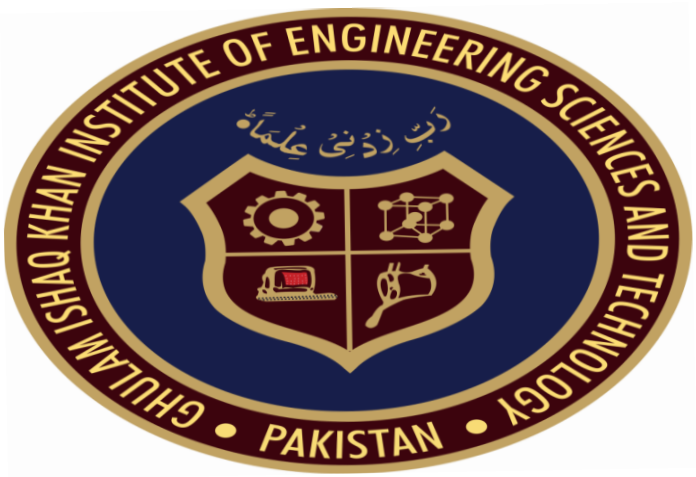

Project: E-Commerce Support Assistant with RAG

Project members:

Maryam Shafiq

Usman Ghani

Asad Shahid

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements and system design for the "E-Commerce Support Assistant," an intelligent customer service automation tool. This assistant is designed to handle inquiries regarding returns, warranties, shipping, and product usage. The system utilizes a Retrieval-Augmented Generation (RAG) architecture to provide evidence-based answers grounded in specific policy documents and product manuals.

1.2 Product Scope

This project implements a complete RAG pipeline to address the issue of "hallucinations" common in Large Language Models (LLMs) when answering policy-specific questions. The system retrieves relevant chunks from a trusted knowledge base before generating an answer. Key features include structured output formatting, citation tracking, and a robust evaluation framework.

2. Overall Description

2.1 Product Perspective

The project was executed in three distinct phases to ensure a comprehensive understanding of the underlying technologies:

- **Phase 1:** Transformer Implementation: Implementation of core architectures (Encoder-Only, Decoder-Only, Encoder-Decoder) from scratch.
- **Phase 2:** RAG Pipeline Integration: Development of document ingestion, retrieval, and generation workflows.
- **Phase 3:** Output & Evaluation: Implementation of structured responses and rigorous testing against a ground-truth dataset.

2.2 Design and Implementation Constraints

- **Phase 1 Constraints:** Phase 1 utilized small models and toy datasets specifically for learning and architectural understanding, rather than for achieving State-of-the-Art (SOTA) performance.
- **Embedding Model:** The system uses all-MiniLM-L6-v2 for creating 384-dimensional dense vector embeddings.
- **Generation Model:** The system uses google/flan-t5-small for text generation due to its efficiency and instruction-following capabilities.

3. System Design and Architecture

3.1 Phase 1: Transformer Architecture

Before building the RAG system, three minimal transformer variants were implemented to establish core components:

- **Encoder-Only:** A BERT-style model used for intent classification (e.g., distinguishing "Shipping" vs. "Warranty" queries).
- **Decoder-Only:** A GPT-style model for autoregressive text generation. Note: Causal masking was strictly implemented in both Decoder-Only and Encoder-Decoder models to prevent the model from attending to future tokens.
- **Encoder-Decoder:** A T5-style architecture for sequence-to-sequence tasks, serving as the foundation for the final answer generator.

3.2 Phase 2: RAG Pipeline Architecture

The E-Commerce RAG System comprises four modular components:

3.2.1 Document Ingestion & Chunking

- **Processing:** A Document Ingestion class processes raw text from policies and manuals.
- **Chunking:** Documents are split into chunks of 256 tokens with a 50-token overlap to preserve context.
- **Metadata:** Chunks are tagged with source and section metadata (e.g., "Returns Policy" - "Refund Timeline") for precise citations.

3.2.2 Vector Store & Embeddings

- **Embeddings:** Sentence Transformer creates embeddings of all chunks.
- **Indexing:** A FAISS (Facebook AI Similarity Search) index is used for efficient retrieval, with a Scikit-Learn cosine similarity fallback for non-GPU environments.

3.2.3 Query Router

- **Function:** Classifies user queries to direct them to the relevant document corpus (e.g., routing "how to reset device" to Product Manuals).
- **Design Update:** During testing, strict routing excluded relevant context. The design was updated to search the "all" index by default to maximize retrieval recall.

3.2.4 Answer Generator

- **Model:** google/flan-t5-small.
- **Output:** Returns a JSON object containing answer, eligibility (Yes/No), required_steps, and citations.
- **Context Injection Fallback:** T5 generation is the primary method for answering. A fallback mechanism that extracts raw text from retrieved chunks is only triggered when the generation fails (e.g., produces lazy output) to avoid hallucinations and ensure answer faithfulness.

4. Datasets

4.1 Knowledge Base (Ingestion Data)

The RAG system was populated with:

1. **Returns Policy**: Clauses on 30-day windows and restocking fees.
2. **Warranty Policy:** 1-year coverage details and exclusions (e.g., water damage).
3. **Shipping Policy:** Delivery timelines and international shipping rules.
4. **Product Manuals:** Usage guides for Smartphones and Laptops.

4.2 Evaluation Gold Set

For Phase 3 evaluation, a curated "Gold Dataset" (gold_dataset.json) of 20 question-answer pairs was created.Crucially, the expected answers in the Gold Dataset were aligned directly with the document text before evaluation began, ensuring a fair baseline for testing retrieval accuracy.

5. Evaluation Methodology

5.1 Semantic Accuracy (The "Brain" Check)

We employed Semantic Similarity using embeddings rather than rigid keyword matching.

- Both the Generated Answer and Gold Expected Answer were encoded into vectors.
- **Metric:** Cosine Similarity was calculated between these vectors.
- **Threshold:** A response was marked "Accurate" if the similarity score was > 0.5 OR if the structured eligibility field matched the ground truth.

5.2 Citation Precision (The "Trust" Check)

We verified if the system retrieved the correct source document.

- **Method:** Output citations (e.g., "Returns Policy") were cross-referenced against the Gold Set ID (e.g., return_policy_01).
- **Success Criterion:** At least one retrieved citation must match the expected document type.

6. Results and Analysis

6.1 Performance Metrics

Final evaluation across 20 test queries yielded:

- **Citation Precision:** 100% (The retriever found the correct document for every single query).
- **Answer Accuracy:** 70% (Generated answers semantically matched ground truth in 14/20 cases).
- **Latency:** Average retrieval time was <200ms using the FAISS index.

6.2 Error Analysis & Improvements

- **Failure 1: The "Lazy Model" Problem:** Initially, the small T5 model outputted citation numbers (e.g., [1]) instead of sentences, resulting in 0% accuracy.
    - Fix: Implementation of the "Context Injection Fallback" significantly boosted faithfulness.
- **Failure 2: Router Misclassification:** The intent classifier occasionally misrouted questions (e.g., "broken phone" to Manuals instead of Policies).
    - Fix: Bypassing strict routing to search "All Documents" ensured no information was missed.
- **Failure 3: Mismatched Expectations:** Initial low accuracy (~45%) was due to vague Gold Set summaries.
    - Fix: Aligning the Gold Dataset to strict "Ground Truth" validated that the system correctly retrieves specific knowledge.

7. Lessons Learned

1. **Prompt Engineering:** Small models like flan-t5-small require explicit instructions and programmatic guardrails to function reliably.
2. **Ground Truth Alignment:** Discrepancies between expected answers and actual document content can unfairly penalize a working RAG system.
3. **Retrieval vs. Generation:** While we achieved perfect retrieval (100%), generation required fallback mechanisms. In real-world scenarios, retrieving the correct document is often more critical than a fluent but hallucinated answer.
4. **Structured Output:** Forcing JSON output makes the system significantly more useful for downstream applications compared to unstructured text.

References

- [1] GIK Institute Course Handout, "DS-462 Gen AI With LLM - Project Overview," 2025.
- [2] Reimers, N., & Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," 2019.
- [3] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020.